Panic, Recover et Résilience

Objectifs

1. Appliquer la convention `Must...` pour les validations critiques au démarrage d'un binaire (*Fail-Fast*).
2. Isoler un crash d'exécution (`panic`) à l'aide du couple **`defer`** / **`recover()`** et le convertir proprement en valeur `error` grâce aux valeurs de retour nommées.
3. Agréger plusieurs erreurs simultanées avec **`errors.Join`** (Go 1.20+) et inspecter l'arbre d'erreurs résultant avec `errors.Is`.
4. Sécuriser la fermeture d'une ressource en écriture (`defer Close()`) sans perdre une erreur de *flush* disque/réseau.

Exercice 1 : Convention `Must...` et bac à sable anti-crash (`recover`)

Vous développez un moteur d'exécution de plugins pour une passerelle API. Certains plugins écrits par des équipes tierces contiennent des bugs graves (déréférencement de pointeur `nil`, dépassement d'indice de slice, division par zéro). Si un plugin plante, la passerelle doit intercepter le crash, le convertir en `error` standard enrichie de sa trace d'appels (*Stack Trace*), et continuer à traiter les requêtes suivantes sans jamais interrompre le processus principal.

1. **Initialisation *Fail-Fast* avec `Must...` :**
   - Écrivez une fonction `MustValiderCleMaitre(cle string)` qui vérifie que `len(cle) >= 16`. Si ce n'est pas le cas, elle déclenche immédiatement un `panic("clé maître invalide : 16 caractères minimum requis")`.
   - Appelez cette fonction au tout début de `main()` avec une clé valide pour autoriser le démarrage du serveur.
2. **Simulation de plugins instables :** Écrivez une fonction `executerPluginTiers(nomPlugin string, payload []byte) string` qui simule différents comportements selon `nomPlugin` :
   - `"plugin-auth"` : retourne `"AUTH_OK"` normalement.
   - `"plugin-nil"` : déclare un pointeur `var ptr *int` et tente d'écrire `*ptr = 42` (provoque un `panic` runtime).
   - `"plugin-slice"` : tente de lire `payload[999]` (provoque un `panic: runtime error: index out of range`).

    - `"plugin-custom"` : appelle explicitement `panic("erreur fatale interne du plugin")`.
3. **Encapsulation sécurisée (`ExecuterEnSecurite`) :**
    - Écrivez une fonction wrapper utilisant des **valeurs de retour nommées** :

```go
func ExecuterEnSecurite(nomPlugin string, payload []byte) (resultat string, err error)
```

    - À l'aide d'une fonction anonyme `defer func() { ... }()` et de `recover()`, interceptez tout `panic` survenant pendant l'appel à `executerPluginTiers`.
    - Lorsqu'un `panic` est capturé, assignez à la variable de retour `err` une erreur formatée contenant le nom du plugin, la cause récupérée par `recover()`, ainsi que la pile d'appels obtenue via `debug.Stack()` (package `"runtime/debug"`).
4. **Validation de la résilience :**
    - Dans `main()`, faites une boucle sur la liste `[]string{"plugin-nil", "plugin-slice", "plugin-custom", "plugin-auth"}` en appelant `ExecuterEnSecurite` pour chacun.
    - Vérifiez que les 3 premiers retournent une `error` proprement gérée et que le 4ème (`"plugin-auth"`) s'exécute avec succès, prouvant que le processus n'a pas crashé !

Exercice 2 : Validateur multi-critères avec `errors.Join`

Lors de la validation d'un fichier de configuration de déploiement Cloud, s'arrêter à la première erreur trouvée est frustrant pour l'administrateur système : il souhaite obtenir la liste **complète** de toutes les anomalies en une seule exécution, tout en permettant au code appelant de tester la présence d'erreurs spécifiques via `errors.Is`.

1. Déclarez 4 erreurs sentinelles :
    - `ErrNomServiceVide`
    - `ErrPortHorsLimites`
    - `ErrMemoireInsuffisante`
    - `ErrReplicasInvalides`
2. Définissez une structure `ConfigDeploiement` avec les champs `NomService string`, `Port int`, `MemoireMo int` et `Replicas int`.
3. Écrivez une fonction `ValiderDeploiement(cfg ConfigDeploiement) error` qui :
    - Vérifie **indépendamment** les 4 règles suivantes (sans faire de `return` anticipé) en accumulant les erreurs dans une slice `var errs []error` :

        1. `NomService != ""` (sinon ajoute `ErrNomServiceVide`).
        2. `Port >= 1024 && Port <= 65535` (sinon ajoute `ErrPortHorsLimites`).
        3. `MemoireMo >= 128` (sinon ajoute `ErrMemoireInsuffisante`).
        4. `Replicas >= 1 && Replicas <= 50` (sinon ajoute `ErrReplicasInvalides`).
    - Retourne le résultat de **`errors.Join(errs...)`**.
4. **Tests et inspection d'arbre d'erreurs :**
    - Testez `ValiderDeploiement` avec une configuration valide : vérifiez que `errors.Join` retourne bien `nil`.
    - Testez avec une configuration violant 3 règles simultanément (`NomService: ""`, `Port: 80`, `MemoireMo: 64`, `Replicas: 3`).
    - Affichez l'erreur jointe complète, puis vérifiez avec **`errors.Is`** que `errors.Is(err, ErrPortHorsLimites)` et `errors.Is(err, ErrMemoireInsuffisante)` renvoient tous les deux `true`, tandis que `errors.Is(err, ErrReplicasInvalides)` renvoie `false`.

Exercice 3 : Fermeture transactionnelle sécurisée (`defer Close()` + `errors.Join`)

En production, ignorer l'erreur retournée par `Close()` lors d'un `defer ressource.Close()` sur un flux en écriture est un piège classique : si une erreur survient à la fois pendant l'écriture **et** pendant la fermeture finale (*flush*), l'une des deux erreurs est habituellement écrasée ou perdue.

1. Créez une structure `TransactionDisqueSimulee` contenant deux booléens de simulation de panne : `EchecEcriture bool` et `EchecFlushClose bool`.
2. Ajoutez-lui deux méthodes :
    - `EcrireBloc(donnees string) error` : retourne `errors.New("erreur I/O : secteur disque défectueux pendant l'écriture")` si `EchecEcriture` est vrai, sinon `nil`.
    - `Close() error` : retourne `errors.New("erreur flush : impossible de synchroniser le tampon lors du Close")` si `EchecFlushClose` est vrai, sinon `nil`.
3. Écrivez une fonction utilisant un **retour d'erreur nommé** :

```
Go
func ExporterJournal(tx *TransactionDisqueSimulee, message string) (errFinale
error)
```

    - Enregistrez un `defer func() { ... }()` qui appelle `tx.Close()` et utilise **`errors.Join(errFinale, errClose)`** pour combiner l'erreur éventuelle de `EcrireBloc` avec l'erreur éventuelle de `Close()` sans jamais en perdre aucune !
    - Appelez ensuite `return tx.EcrireBloc(message)`.
4. Dans `main()`, testez les 4 combinaisons possibles (`false/false`, `true/false`, `false/true`, et surtout `true/true` où les deux pannes surviennent simultanément) et vérifiez que dans le 4ème cas, `errFinale` contient bien les deux erreurs fusionnées.